Stateless vs Stateful

sessions, sticky sessions and scaling WebSockets

Watch the 3-min video

In one line: stateless services keep nothing between requests, so any instance can serve anyone and they scale freely; push state into a few purpose-built stateful systems that replicate and rebalance.

1 Two kinds of components



Figure 1. Stateless components are interchangeable; stateful ones hold data or connections that live on a specific node.

Stateless services scale easily: add instances, remove instances, lose one in a crash, and nothing is lost. The load balancer can send any request anywhere, and autoscaling just works.

2 Making a service stateless

FASTAPI SESSIONS

The app keeps sessions in an in-memory dictionary. With two pods, half the requests land on the pod that doesn't know the user. Move sessions to Redis (or use signed tokens like JWTs), and every pod becomes interchangeable.

Option	How	Trade-off
Shared session store	Sessions in Redis, keyed by a cookie	One more system to run (make it redundant)
Signed tokens	State inside a signed JWT cookie	No store, but hard to revoke; keep tokens small
Sticky sessions	Load balancer pins a user to one server	Quick fix; uneven load, sessions lost when that server dies

3 What stateful systems need

- **Placement:** which node owns which data or which user (e.g. consistent hashing).
- **Replication:** copies, so a node can die safely.
- **Rebalancing:** moving data or connections when nodes join or leave.

4 Scaling a chat app's WebSockets

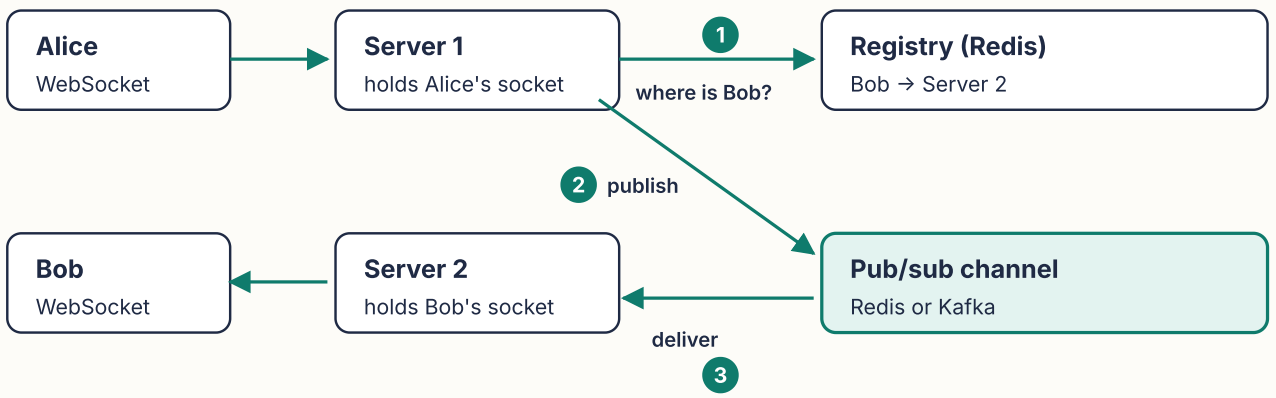


Figure 2. Alice and Bob are connected to different servers. A registry finds Bob's server; pub/sub carries the message there.

1. Each user holds a long-lived WebSocket to one server; the server records *user → server* in a registry with a TTL heartbeat.
2. When Alice sends a message, Server 1 looks up Bob and publishes to Server 2's channel (or a per-user/per-room channel).
3. Server 2 pushes it down Bob's socket. Messages are also saved to the database, so a client that reconnects can fetch what it missed.
4. If a server dies, its clients reconnect (with backoff and jitter) to another server, which updates the registry.

THE RULE OF THUMB

Keep compute stateless, and push state into a few systems built to replicate and rebalance: databases, caches, message brokers. Scale the easy parts freely and the hard parts carefully.

COMMON MISTAKES

- In-memory sessions on horizontally scaled servers.
- Sticky sessions as the long-term answer.
- Broadcasting every chat message to every server.
- Reconnect storms: all clients reconnecting at once with no jitter.

5 Key terms

Stateless: nothing kept between requests

Sticky session: LB pins a user to one server

Pub/sub: publish once, subscribers receive

Stateful: data or connections live on a node

JWT: a signed token that carries state

Rebalancing: moving data when nodes change

6 Interview questions

Q1. Why are stateless services easier to scale? BASIC

Any instance can serve any request, so you can add, remove or lose instances freely and the load balancer can route anywhere.

Q2. How would you scale a stateful WebSocket server? BASIC

Add servers behind a load balancer, track which user is on which server, route messages between servers with pub/sub, and let clients reconnect on failure.

Q3. What do stateful systems need that stateless ones don't? BASIC

Placement (who owns what), replication (survive node loss) and rebalancing (move data when nodes change).

Q4. After adding a second server, some chat messages never arrive. Why? SCENARIO

The sender's server only knows its own sockets; the recipient is on another server. Add a user→server registry and pub/sub between servers.

Q5. Sessions in Redis or JWTs? SCENARIO

Redis: easy revocation and small cookies, but a dependency to keep redundant. JWTs: no store, but revocation is hard and tokens grow. Many systems use short-lived JWTs plus a refresh token in a store.

Q6. A WebSocket server with 100K connections crashes. What happens next? SCENARIO

Its clients reconnect to other servers; use exponential backoff with jitter to avoid a stampede, update the registry, and fetch missed messages from the database.

Q7. When are sticky sessions acceptable? TRICKY

As an optimisation (better cache locality) when losing the session is harmless, or briefly during a migration; not as the only place state lives.

Q8. Is a cache stateful? TRICKY

Yes, it holds data, but it's usually treated as disposable: losing it should only cost performance (cache misses), never correctness. Plan for cold-start load on the database.
